1.FWHT

```
void fwht(vector<Z>& a, auto combine) {
    int n = a.size();
    for(int len = 1; len < n; len <<= 1) {
        for(int i = 0; i < n; i += len << 1) {
            for(int j = 0; j < len; j++) {
                combine(a[i + j], a[i + j + len]);
            }
        }
    }
}

void work(int n, vector<Z> a, vector<Z> b, int type) {
    int N = 1 << n;
    if(type == 0) {
        auto f = [](Z& x, Z& y) { y += x; };
        auto g = [](Z& x, Z& y) { y -= x; };

        fwht(a, f);
        fwht(b, f);

        for(int i = 0; i < N; i++) {
            a[i] *= b[i];
        }

        fwht(a, g);
    } else if(type == 1) {
        auto f = [](Z& x, Z& y) { x += y; };
        auto g = [](Z& x, Z& y) { x -= y; };

        fwht(a, f);
        fwht(b, f);

        for(int i = 0; i < N; i++) {
            a[i] *= b[i];
        }

        fwht(a, g);
    } else {
        auto f = [](Z& x, Z& y) {
            Z u = x, v = y;
            x = u + v;
            y = u - v;
        };
        auto g = [](Z& x, Z& y) {
            Z u = x, v = y;
            x = (u + v) * inv2;
            y = (u - v) * inv2;
        };
```

```
        fwht(a, f);
        fwht(b, f);

        for(int i = 0; i < N; i++) {
            a[i] *= b[i];
        }

        fwht(a, g);
    }

    for(int i = 0; i < N; i++) {
        cout << a[i] << " \n"[i == N - 1];
    }
}
```

2.杜教筛

```
constexpr int N = 2000000;

i64 phi[N + 1];
i64 mu[N + 1];
int minp[N + 1];
vector<int> primes;

void init() {
    phi[1] = 1;
    mu[1] = 1;

    for(int i = 2; i <= N; i++) {
        if(!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
            mu[i] = -1;
        }
        for(auto p : primes) {
            if(i * p > N) break;
            minp[i * p] = p;
            if(minp[i] == p) {
                phi[i * p] = phi[i] * p;
                mu[i * p] = 0;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
                mu[i * p] = -mu[i];
            }
        }
    }

    for(int i = 2; i <= N; i++) {
```

```
        phi[i] += phi[i - 1];
        mu[i] += mu[i - 1];
    }

};

map<i64, pair<i64, i64>> memo;

pair<i64,i64> get(i64 n) {
    if(n <= N) {
        return {phi[n], mu[n]};
    }
    auto it = memo.find(n);
    if(it != memo.end()) {
        return it->second;
    }

    i64 sum_phi = n * (n + 1) / 2;
    i64 sum_mu = 1;
    for(i64 L = 2, R; L <= n; L = R + 1) {
        R = n / (n / L);
        auto [x, y] = get(n / L);
        sum_phi -= x * (R - L + 1);
        sum_mu -= y * (R - L + 1);
    }

    return memo[n] = {sum_phi, sum_mu};
}

void solve() {
    int n;
    cin >> n;

    auto [sum_phi, sum_mu] = get(n);

    cout << sum_phi << " " << sum_mu << "\n";
}
```

3.min_25筛

```
vector<int> minp, primes;
vector<Z> sp0, sp1;
void init(int n) {
    minp.resize(n + 1);
    sp0.resize(n + 1);
    sp1.resize(n + 1);
    for(int i = 2; i <= n; i++) {
        if(!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
```

```
            int j = primes.size();
            sp0[j] = sp0[j - 1] + 1;
            sp1[j] = sp1[j - 1] + i;
        }
        for(auto p : primes) {
            if(i * p > n) break;
            minp[i * p] = p;
            if(minp[i] == p) break;
        }
    }
}

void solve() {
    i64 n;
    cin >> n;
    int sq = sqrt(n);
    init(sq);

    vector<i64> w;
    vector<int> id1(sq + 1), id2(sq + 1);
    for(i64 L = 1, R; L <= n; L = R + 1) {
        R = n / (n / L);
        i64 v = n / L;
        w.push_back(v);
        if(v <= sq) id1[v] = w.size() - 1;
        else id2[R] = w.size() - 1;
    }

    vector<Z> g0(w.size()), g1(w.size());
    for(int i = 0; i < w.size(); i++) {
        Z x = w[i];
        g0[i] = x - 1;
        g1[i] = x * (x + 1) * inv2 - 1;
    }

    auto get = [&](i64 x) {
        return x <= sq ? id1[x] : id2[n / x];
    };

    for(int j = 0; j < primes.size(); j++) {
        i64 p = primes[j];
        i64 p2 = p * p;
        for(int i = 0; i < w.size(); i++) {
            if(w[i] < p2) break;
            int k = get(w[i] / p);
            g0[i] -= g0[k] - sp0[j];
            g1[i] -= p * (g1[k] - sp1[j]);
        }
    }

    auto S = [&](auto&& self, i64 x, int j) -> Z {
        if(x <= 1 || j < primes.size() && primes[j] > x) return 0;
```

```
        int k = get(x);

        Z ans = g1[k] - g0[k] + 2;
        if(j >= 1) {
            ans -= sp1[j] - sp0[j] + 2;
        }

        for(int i = j; i < primes.size(); i++) {
            i64 p = primes[i];
            if(p * p > x) break;
            for(i64 pe = p, e = 1; pe * p <= x; pe *= p, e++) {
                Z cur = Z(p ^ e);
                ans += cur * self(self, x / pe, i + 1);
                ans += Z(p ^ (e + 1));
            }
        }

        return ans;
    };

    cout << S(S, n, 0) + 1 << "\n";
}
```

4.可持久化线段树

```
template<class Info>
struct PersistentSegmentTree {
    struct Node {
        int l = 0, r = 0;
        Info info;
    };
    int n;
    vector<Node> nodes;

    PersistentSegmentTree() : n(0) {}
    PersistentSegmentTree(int n_, int q = 0) {
        init(n_, q);
    }

    void init(int n_, int q = 0) {
        n = n_;
        nodes.clear();
        nodes.reserve(4 * n + (q > 0 ? q * (__lg(n) + 2) : 0));
        nodes.push_back({0, 0, Info()});
    }

    //下标从1开始
    template<class T>
    int build(const vector<T>& init_) {
        auto dfs = [&](auto &&self, int l, int r) -> int {
```

```
            int u = newNode();
            if(l == r) {
                if(l < (int)init_.size()) nodes[u].info = init_[l];
                return u;
            }
            int mid = l + r >> 1;
            nodes[u].l = self(self, l, mid);
            nodes[u].r = self(self, mid + 1, r);
            pull(u);
            return u;
        };
        return dfs(dfs, 1, n);
    }
    int build() {
        auto dfs = [&](auto &&self, int l, int r) -> int {
            int u = newNode();
            if(l == r) return u;
            int mid = l + r >> 1;
            nodes[u].l = self(self, l, mid);
            nodes[u].r = self(self, mid + 1, r);
            pull(u);
            return u;
        };
        return dfs(dfs, 1, n);
    }
    int newNode() {
        nodes.push_back({0, 0, Info()});
        return nodes.size() - 1;
    }
    int copyNode(int src) {
        Node tmp = nodes[src];
        nodes.push_back(tmp);
        return nodes.size() - 1;
    }
    void pull(int u) {
        if(u) nodes[u].info = nodes[nodes[u].l].info + nodes[nodes[u].r].info;
    }
    int modify(int prev, int l, int r, int x, const Info& v) {
        int u = copyNode(prev);
        if(l == r) {
            nodes[u].info = v;

            //需要累加时这样写
            //nodes[u].info = nodes[u].info + v;
            return u;
        }
        int mid = l + r >> 1;
        if(x <= mid) {
            nodes[u].l = modify(nodes[prev].l, l, mid, x, v);
        } else {
            nodes[u].r = modify(nodes[prev].r, mid + 1, r, x, v);
        }
        pull(u);
```

```
        return u;
    }
    int modify(int prev, int x, const Info& v) {
        return modify(prev, 1, n, x, v);
    }
    Info query(int u, int l, int r, int x, int y) {
        if(!u) return Info();
        if(x <= l && r <= y) {
            return nodes[u].info;
        }
        int mid = l + r >> 1;
        if(y <= mid) {
            return query(nodes[u].l, l, mid, x, y);
        }
        if(x > mid) {
            return query(nodes[u].r, mid + 1, r, x, y);
        }
        return query(nodes[u].l, l, mid, x, y) + query(nodes[u].r, mid + 1, r, x, y);
    }
    Info query(int u, int x, int y) {
        if(y < x) {
            return Info();
        }
        return query(u, 1, n, x, y);
    }
    template<class F>
    int findFirst(int u, int l, int r, int x, int y, F&& pred) {
        if(l > y || r < x || u == 0 || !pred(nodes[u].info)) {
            return -1;
        }
        if(l == r) {
            return l;
        }
        int mid = l + r >> 1;
        int res = findFirst(nodes[u].l, l, mid, x, y, pred);
        if(res == -1) {
            res = findFirst(nodes[u].r, mid + 1, r, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findFirst(int u, int x, int y, F&& pred) {
        return findFirst(u, 1, n, x, y, pred);
    }
};

struct Info {
    int cnt = 0;
};

Info operator+(const Info& a, const Info& b) {
    return {a.cnt + b.cnt};
}
```